Ransomware

Group 2

Danny Lo, Michael Mendoza,

Leon Do, Fortune Meya, Arturo Flores, Anthony Canchola

Professor Louis Uuh

California State University, Long Beach

College of Engineering

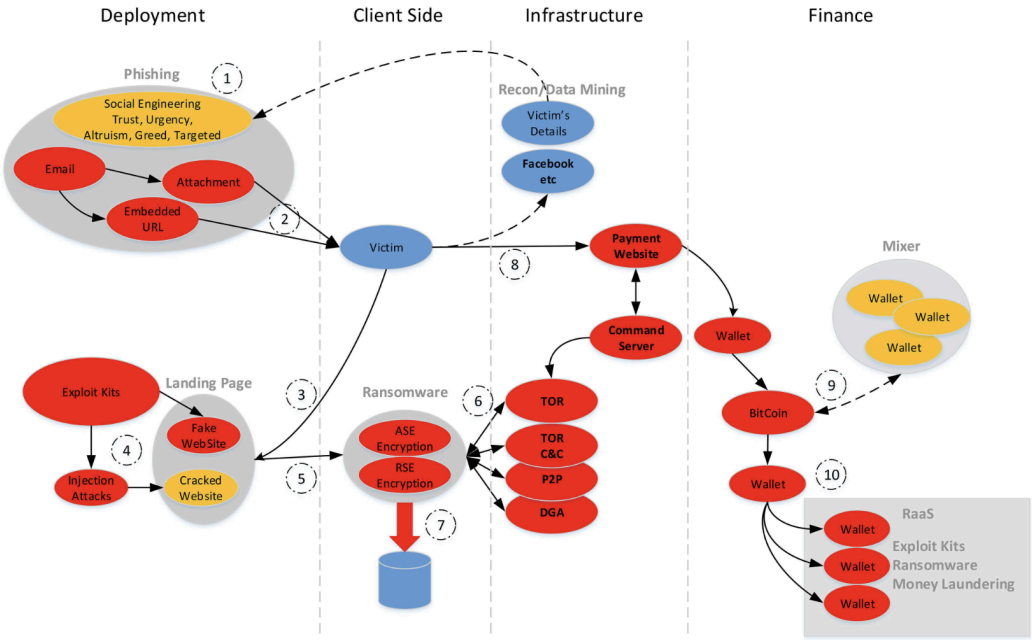
CECS378, Sec 02 4812, Spring 2026

Ransomware

Ransomware is a type of malicious software that is used to block access to a system or encrypt a user's files, then demand a payment (ransom) to restore access. It is mainly used by cybercriminals to make money by targeting individuals, businesses, or organizations.

A basic ransomware attack usually starts when a user unknowingly downloads or opens something malicious, often through phishing emails, fake links, or infected software. Once inside the system, the ransomware installs itself and begins encrypting files so they can't be accessed. After the encryption is complete, the attacker displays a message demanding payment, often in cryptocurrency, in exchange for a decryption key.

Overall, ransomware is dangerous because it prevents users from accessing important data and can cause serious financial and operational damage.



Implementation

Github: https://github.com/yushasama/larpcheck/tree/main

Overview: The Program, LarpCheck, is a localized ransomware simulation developed in Python. The system incorporates three independent security layers: file encryption using AES-GCM, encrypted test case protection using Fernet with scrypt-based key derivation, and an HMAC-based token system to prevent tampering or memory manipulation. Encrypted files are structured using a header, nonce, and authenticated ciphertext. To recover access, the victim must complete three Data Structures & Algorithms (DSA) programming challenges. The system is designed to be entirely self-contained, requiring no external network connectivity, which minimizes the attack surface and ensures it can be safely demonstrated in a controlled environment.

Tactics: To mimic real-world malware, LarpCheck implements basic anti-debugging techniques. Upon execution, the __main__ entry point invokes a security check to determine if the process is being monitored by a debugger. It utilizes sys.gettrace() to detect if a Python debugger is attached. On Windows environments, it makes a direct call to the WinAPI via ctypes.windll.kernel32.IsDebuggerPresent(). On macOS and Linux environments, the detection relies solely on sys.gettrace(), as the WinAPI is unavailable on non-Windows systems. If any monitoring is detected, the program terminates immediately with sys.exit(1), aiming to make step-by-step runtime analysis more difficult by detecting active debuggers and exiting early.

Cryptographic Implementation: The program encrypts the files within a designated directory (sandbox/). We use AES-GCM as our primary choice of encryption and each file targeted is encrypted in-place. For protecting internal test cases we utilize the scrypt key derivation

function. This transforms a secret passphrase into a 32-byte key, which is then used by a Fernet (AES-128-CBC + HMAC-SHA256) implementation to ensure that not just anyone can simply read the expected answers from the disk.

Fernet is an implementation of AES-128 in CBC mode with an HMAC-SHA256 signature. This ensures that the test cases are not only encrypted but also have integrity. This prevents straightforward modification of test inputs to make the challenges easier. The key for this is never stored and is instead derived at runtime using scrypt, a memory-hard key derivation function that prevents brute-force attacks on our internal password.

Integrity Protection: The most critical defense against cheating the simulation is the HMAC-SHA256 solve-token system. The system requires a cryptographic signature to prevent others from tampering with the answers. When a user provides a correct solution to a problem the system generates a unique token (Token = HMAC-SHA256(secret Key, Problem ID))

Security Considerations: The system is designed to mitigate several common attack vectors,

- Hex Editing / File Inspection
    - Encrypted test case outputs & sandbox files are protected using AES-GCM and Fernet. AES-GCM provides authenticated encryption, while Fernet achieves confidentiality and integrity through a combination of AES-CBC and HMAC-SHA256.
- Memory Patching
    - Attempts to modify in-memory variables (an example would be marking problems as solved) are mitigated by the HMAC-based token system,

requiring a valid cryptographic signature for each solved problem.

- State File Tampering
    - Editing the local state file to fake progress is discouraged, as all solved states are verified using HMAC tokens.
- Debugging / Reverse Engineering
    - The program includes basic anti-debugging checks, aiming to limit runtime inspection of the encryption logic.
- File Restoration Attacks:
    - Modifications to encrypted files are detectable due to AES-GCM's integrity checks, reducing the effectiveness of manual file restoration or tampering attempts.

The use of HMAC-SHA256 tokens is our main defense against runtime memory manipulation. A user might use a tool like cheat engine to change a solved variable from 0 to 1 in a computer's RAM. However, LarpCheck requires a cryptographic signature (the HMAC token) for every solved problem. Since the attacker does not know the secret key used to sign these tokens they cannot forge a valid solve state in memory and the decryption process will fail.

During the final decryption phase the decrypt sandbox() function iterates through every solved problem and verifies these tokens if any token is missing or incorrect, the program detects a tampered state and refuses to release the files. This prevents bypassing the challenge by editing the local state file.

The ransom is processed by an internal judge that executes user submissions against encrypted inputs and compares the resulting output in memory. This ensures that the plaintext answers to

the challenge never touch the disk.

Challenges

One of the primary challenges identified during the development of the LarpCheck is the fundamental nature of the ransom. In standard ransomware scenarios, the unlock condition is generally in the form of a financial transaction (typically Bitcoin), which cannot be bypassed without the transfer.

In our simulation, the unlock condition is solving the programming problems. This is done as a safe way of showcasing how we could implement a ransomware attack without having it be damaging when showcasing/testing.

The main challenge was maintaining a realistic attack while ensuring the safety of our systems. Asides from using Virtual Machines, we decided to also focus on two aspects as a result, sandbox isolation and a non-destructive rest. In a real attack ransomware would traverse the entire filesystem or mapped network drives. To ensure this project remained ethical and more importantly non-destructive, we strictly limited the scope to a sandbox directory and implemented a sandbox_backup feature. While real ransomware (like WannaCry) often threatens to delete files after a timer expires, we decided to omit the data deletion feature and maintain persistence of the cipher text to allow for repeated testing and demonstration.

Roles

- Danny Lo: For this project, I helped review the ransomware code, helping set up the project document and presentation slides, writing the introduction for the report, and completing several slides for the final presentation.

- Michael Mendoza: Peer reviewed document to fix flow and helped with implementation portion, demo portion of Slides
- Anthony Canchola: Helped write the rough draft of the document (Implementation and Challenges portion), Did Introduction to Ransomware and Example of Ransomware: Wannacry slides
- Leon Do: Wrote the codebase, Document
- Fortune Meya: Conclusion part of the slides, Helped with the document
- Arturo Flores: Conclusion portion of the slides. Helped review the document.

Citation

Cryptography Developers. (n.d.-a). Authenticated encryption. Cryptography. https://cryptography.io/en/latest/hazmat/primitives/aead/

Cryptography Developers. (n.d.-b). Fernet (symmetric encryption). Cryptography. https://cryptography.io/en/stable/fernet/

O'Kane, Philip, et al. "Evolution of Ransomware." IET Networks, vol. 7, no. 5, Sept. 2018, pp. 321–327, doi:10.1049/iet-net.2017.0207.

Python Software Foundation. (n.d.-a). hashlib — Secure hashes and message digests. Python documentation. https://docs.python.org/3/library/hashlib.html#hashlib.scrypt

Python Software Foundation. (n.d.-b). hmac — Keyed-hashing for message authentication. Python documentation. https://docs.python.org/3/library/hmac.html

Python Software Foundation. (n.d.-c). subprocess — Subprocess management. Python documentation. https://docs.python.org/3/library/subprocess.html

Python Software Foundation. (n.d.-d). tempfile — Generate temporary files and directories. Python documentation. https://docs.python.org/3/library/tempfile.html